

Arrays in Java :

What is a Variable in java ?

A variable is a name given for the memory location.

What is the drawback of an ordinary variable ?

An Ordinary variable can hold only one value at a time in random memory location.

Example :

```
int x = 10; //Valid
int y = 90,67; //Invalid
```

We want to store multiple values in a contiguous memory location then we should use another concept called Array.

What is Array in java :

An array is a container object that holds a fixed number of values of a single type in ordered collection.

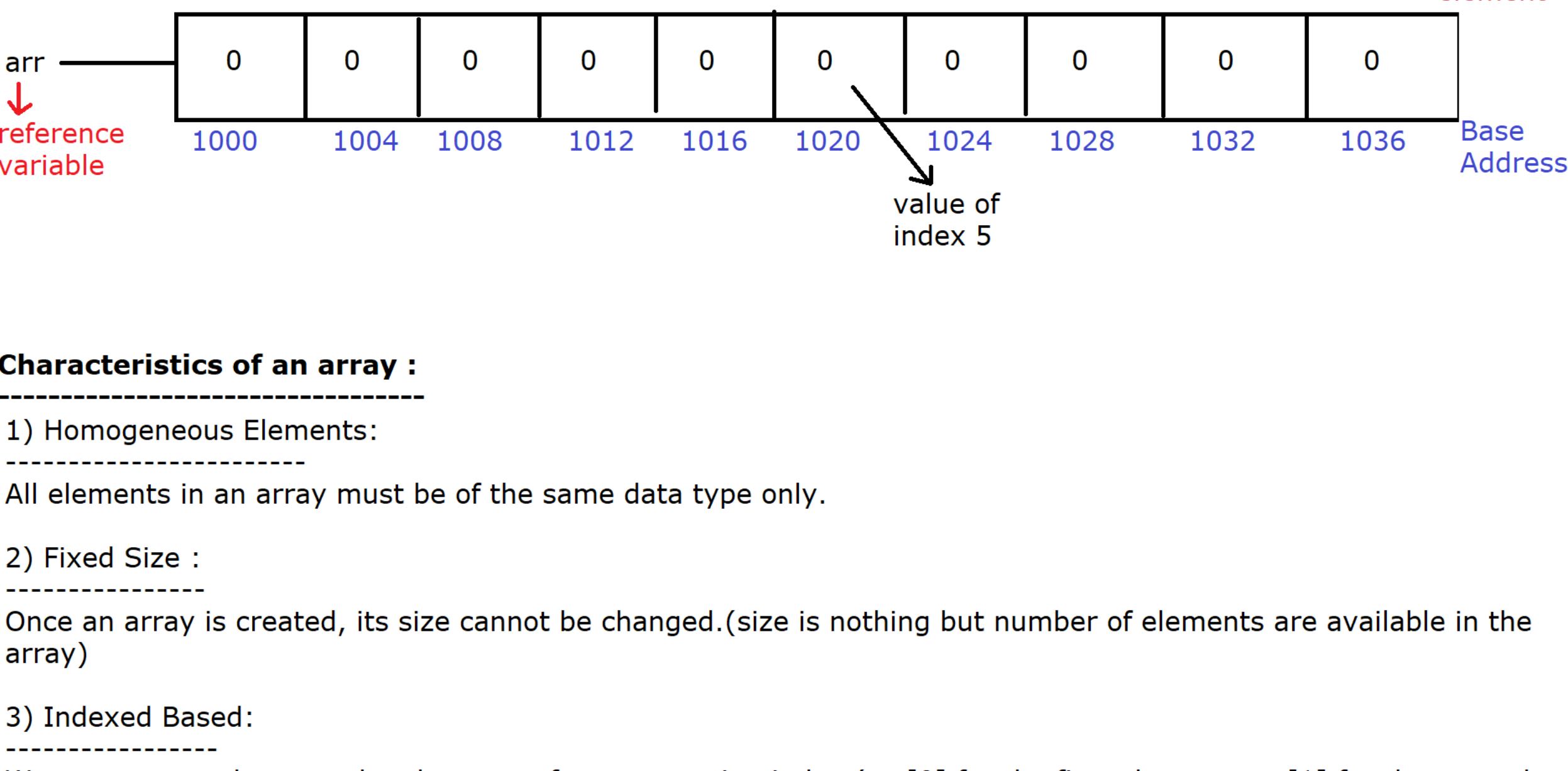
It can store multiple values in a single variable.

Example :-

```
int []x = {12,78,45,34,90}; //Homogeneous
Object [] obj = {12,"Ravi",null, 90.56}; //Object type [Homogeneous i.e. Object type]

The length of an array is established when the array is created. After creation, its length is fixed.
```

In Java array is an object so it is created at HEAP memory at runtime which can hold primitive types as well as reference type.



Characteristics of an array :

- 1) Homogeneous Elements:
All elements in an array must be of the same data type only.
- 2) Fixed Size :
Once an array is created, its size cannot be changed.(size is nothing but number of elements are available in the array)
- 3) Indexed Based:
We can store and access the elements of an array using index (arr[0] for the first element, arr[1] for the second, etc.)
 - a) How to find first and last index position element
int []arr = {10,20,30,40,90,12}; //arr -> 1000x
IO.println("First Element :"+arr[0]); //10
IO.println("Last Element :"+arr[arr.length -1]); //12
 - b) [Why array index starts from 0] [int is taking 4 bytes of Memory]
1000 + (0 * 4) = 1000x //1000 is Base address, 0 is index
1000 + (1 * 4) = 1004x & 4 is the size
1000 + (2 * 4) = 1008x
- 4) Stored in Contiguous Memory:
Array elements are stored in adjacent memory locations, improving access speed (Cache Memory).
- 5) Stored in Heap Memory:
Arrays are objects in Java and are stored in the heap, even if declared inside a method.
- 6) Default Values:
If an array is declared but not initialized, Java assigns default values
- 7) Fast Retrieval:
Direct access to elements using an index provides fast data retrieval (O(1) time complexity).

Drawback of an array :

- 1) Fixed in size
- 2) Insertion OR deletion at the begning or middle is O(n) time complexity
- 3) No predefined method support. [Need to write the logic using Brute force approach]

In Java, Basically there are two types of arrays:

- 1) Single OR One Dimensional Array.([])
- 2) 2D OR Multi-Dimensional Array.([][])

Note :We have jagged array which is a multi-dimensional array where each row can have a different number of columns.

Working with single OR one Dimensional Array :

In Java we can create single dimensional array by using following ways :

Example 1 :

```
int[] arr = new int[5]; //Array of primitive
Student[] students = new Student[10]; //array of reference
```

Note : Here the array memory locations will be initialized with default values.

Example 2 :

Another way to create an array object using new keyword with Inline Initialization [We can initialize the array directly while creating it.]

```
int[] arr = new int[]{10, 20, 30, 40, 50};
```

Note : Here instead of default value user provides the value explicitly.

In the above example, expression is duplicate because the type of array we know from the left side of equal sign and since we are specifying the value of array hence we know the size also.

Instead of this we can use more convenient way which is known as Anonymous array.

Example 3 :

```
int[] arr = {100, 200, 300};
```

Note : This is called anonymous array because we don't specify type and size.

Finally, We can use [] before or after the reference variable name, providing space is optional. The following 5 statements are same

```
int[] num1;
int [] num2;
int []num3;
int num4[];
int num5 [];
```

* An array size must be a positive integer.

```
Example :
void main()
{
    int size = 10;
    int []arr = new int[Integer.MAX_VALUE]; //Maximum size of array
}
```

* JVM generated class for arrays.

The JNI uses the Java VM's representation of type signatures. The following table shows these type signatures.

Java VM Type Signatures	
Type Signature	Java Type
Z	boolean
B	byte
C	char
S	short
I	int
J	long
F	float
D	double
L fully-qualified-class ;	fully-qualified-class
[type	type[]
(arg-types) ret-type	method type

```
package com.ravi.one_dimensional_basic;

record Student() {}

public class JVMGeneratedClass {

    public static void main(String[] args)
    {
        byte b[] = new byte[5];
        IO.println("Class Name is :"+b.getClass().getName());
        IO.println("Super class Name is :"+b.getClass().getSuperClass());

        short s[] = new short[5];
        IO.println("Class Name is :"+s.getClass().getName());

        int i[] = new int[5];
        IO.println("Class Name is :"+i.getClass().getName());

        long l[] = new long[5];
        IO.println("Class Name is :"+l.getClass().getName()); //[]

        float f[] = new float[5];
        IO.println("Class Name is :"+f.getClass().getName());

        double d[] = new double[5];
        IO.println("Class Name is :"+d.getClass().getName());

        char ch[] = new char[5];
        IO.println("Class Name is :"+ch.getClass().getName());

        boolean bi[] = new boolean[5];
        IO.println("Class Name is :"+bi.getClass().getName()); //[Z

        String []arr = new String[5];
        IO.println("Class Name is :"+arr.getClass().getName()); //[]L FQN

        Student []students = new Student[2];
        IO.println("Class Name is :"+students.getClass().getName());
    }
}
```